

- For slow operations that take more than a second or two—for example, downloading or uploading a file
- For operations that have side effects, such as changes to other attributes or objects
- For operations that require additional arguments to be passed to the get or set operation—for example, in a method call like

```
emailObj.getFileAttachment(filename)
```

Programmers often think of methods as verbs (in the sense that methods perform some action), and they think of attributes and properties as nouns (in the sense that they represent some item or object). If your code seems to be performing more of an action of getting or setting rather than getting or setting an item, it might be best to use a getter or setter method. Ultimately, this decision depends on what sounds right to you as the programmer.

The great advantage of using Python’s properties is that you don’t have to use them when you first create your class. You can use regular attributes, and if you need properties later, you can convert the attributes to properties without breaking any code outside the class. When we make a property with the attribute’s name, we can rename the attribute using a prefix underscore and our program will still work as it did before.

Python’s Dunder Methods

Python has several special method names that begin and end with double underscores, abbreviated as *dunder*. These methods are called *dunder methods*, *special methods*, or *magic methods*. You’re already familiar with the `__init__()` dunder method name, but Python has several more. We often use them for *operator overloading*—that is, adding custom behaviors that allow us to use objects of our classes with Python operators, such as `+` or `>=`. Other dunder methods let objects of our classes work with Python’s built-in functions, such as `len()` or `repr()`.

As with `__init__()` or the getter, setter, and deleter methods for properties, you almost never call dunder methods directly. Python calls them behind the scenes when you use the objects with operators or built-in functions. For example, if you create a method named `__len__()` or `__repr__()` for your class,